Chaitin–Gödel I in BRA4: a uniform-in-w contradiction map

1 Chaitin's argument, recalled

Chaitin's proof of Gödel's first incompleteness theorem [1, 2] runs as follows. Fix a sound theory T that interprets enough arithmetic, and fix a universal partial computable interpreter U with a Kolmogorov-complexity bound $K_U(x) := \min\{|p| : U(p) \downarrow= x\}$. There is a constant c (depending only on T and U) such that, for every $L^\star$ and every integer x',

$$T \vdash K_U(\underline{x'}) > L^\star \quad \Longrightarrow \quad T \text{ is inconsistent, provided } L^\star > c + \log L^\star.$$

The contradiction is exhibited by a single short program $g_{L^\star}$: enumerate $\mathsf{thmT}(0), \mathsf{thmT}(1), \ldots$ until a proof of the form $\mathsf{code}K_U(\underline{?}) > L^\star$ is found, then output the witnessed integer x'. The size of $g_{L^\star}$ is $c + \log L^\star$, strictly below $L^\star$, so the very witness x' that T proves to be K_U-incompressible is itself $g_{L^\star}$-output — a contradiction to the size lemma internal to T.

2 The BRA4 formalisation: three layers

2.1 The core: CGI_core_num_raw

The shipped core theorem (commit `2fce05d`, `BRA4/ChaitinG1CoreNumRaw.agda`) is:

$$\begin{aligned}\mathsf{CGI\_core\_num\_raw} : (w\ x : \mathrm{Term}) \to\ & \\ & \mathsf{Deriv}\,(\mathsf{eqF}\,(\mathsf{thmT}\,w)\,(\mathsf{Kcode}_{L^\star}\,x)) \to \\ & \Sigma\,\mathrm{Term}\,(\lambda z.\,\mathsf{Deriv}\,(\mathsf{eqF}\,(\mathsf{thmT}\,z)\,\mathsf{codeFalse})).\end{aligned}$$

No closure hypothesis on w, no isNat, no Con(T). The "small program" $g_{L^\star}$ is realised by the closed BRA function `gLcodeDef` $L^\star$; the search by `FirstHit.Search.gRec`; the universal interpreter by `evalU`; the chain of operational steps closing evalU on `gLcodeDef`'s mu-loop by `ChainKdef.dEval_witness`; the size atom by `dLenStarDef`. The output $z(w)$ is

$$\mathsf{cmp}\,(\mathsf{cmp}\,(\mathsf{exfProof}\,(\mathsf{D}\,g_L\,n_0\,x')\,\mathsf{codeFalse})\,\mathsf{cPos})\,(\mathsf{cmp}\,\mathsf{outerWrap}\,\mathsf{cSizeProofDef}),$$

i.e. a literal encoded modus-ponens / ex-falso assembly.

2.2 The x-free form: CGI_core_num_raw_self

Whether $\mathsf{thmT}\,w$ has the form $\mathsf{Kcode}_{L^\star}\,(\_)$ is read off by the BRA function $\mathsf{outKdef}_{L^\star} = \mathsf{decode} \circ \mathsf{projKdef} \circ \mathsf{thmT}$. Hence the parameter x in the hypothesis is redundant: it can only be $\mathsf{outKdef}_{L^\star}\,w$.

The self-referential form, shipped in `BRA4/ChaitinG1CoreNumRawSelf.agda`, is

$$\begin{aligned}\mathsf{CGI\_core\_num\_raw\_self} : (w : \mathrm{Term}) \to\\ &\mathsf{Deriv}\,(\mathsf{eqF}\,(\mathsf{thmT}\,w)\,(\mathsf{Kcode}_{L^\star}\,(\mathsf{outKdef}_{L^\star}\,w))) \to\\ &\Sigma\,\mathrm{Term}\,(\lambda z.\,\mathsf{Deriv}\,(\mathsf{eqF}\,(\mathsf{thmT}\,z)\,\mathsf{codeFalse})),\end{aligned}$$

and the proof is one line: $\mathsf{CGI\_core\_num\_raw\_self}\,w\,h := \mathsf{CGI\_core\_num\_raw}\,w\,(\mathsf{outKdef}_{L^\star}\,w)\,h$.

2.3 The decomposition: cgFun and cgFalse

The Σ-payload above naturally factors as

$\mathsf{cgFun}_* : (w : \mathrm{Term}) \to \mathsf{Deriv}(\dots) \to \mathrm{Term}$, $\mathsf{cgFalse}_* : (w : \mathrm{Term})\,(d : \dots) \to \mathsf{Deriv}(\mathsf{eqF}\,(\mathsf{thmT}\,(\mathsf{cgFun}_*\,w\,d))\,\mathsf{cod}$

the two field projections of Σ. The first observation is that the produced *Term* is in fact d-independent: every Term passing through `DischargeKdef` / `ChainKdef` / `cgiClash` is built from $\mathsf{closeW}\,w$ and a finite set of constants (the BRA combinators `gRec`, `predFlipDef` $L^\star$, `outKdef` $L^\star$, `runProg`, `fuelMu_fun`, `fuelG`, `thm12_Fun2 runProg`, and the tag-/pi-/sb-encoding constants). This is because $\mathsf{leastNumber}\,B\,\mathit{witness} = \mathsf{mkLeast}\,(\mathsf{g}\,(s\,B))\,\dots$ discards the witness for its first component (the Term $k_{\max}$), and the witness Derivs are only used to construct the proof half *isPf*, not the Term *pf*.

The shipped file `BRA4/CgFun.agda` therefore exposes a uniform-in-w meta-Agda function

$$\mathsf{cgFun} : \mathrm{Term} \to \mathrm{Term}$$

defined by replicating, in a single let-chain, the construction

$$\begin{aligned}cw &:= \mathsf{closeW}\,w\\ k_{\max} &:= \mathsf{gRec}\,O\,(s\,cw)\\ x' &:= \mathsf{outKdef}_{L^\star}\,k_{\max}\\ n_T &:= (\text{sigma-pile fuel in } k_{\max},\ \mathsf{fuelMu\_fun}\,k_{\max}\,k_{\max},\ \mathsf{fuelG}\,k_{\max}\,O)\\ \mathsf{cgFun}\,w &:= \mathsf{cmp}\,(\mathsf{cmp}\,(\mathsf{exfProof}\,D\,\mathsf{codeFalse})\,\mathsf{cPos})\,(\mathsf{cmp}\,\mathsf{outerWrap}\,\mathsf{cSizeProofDef}),\end{aligned}$$

where D, cPos, outerWrap are the let-bound syntactic functions of $k_{\max}$, n_T, x' documented inside the file. The companion theorem is

$$\begin{aligned}\mathsf{cgFalse} : (w : \mathrm{Term}) \to\\ &\mathsf{Deriv}\,(\mathsf{eqF}\,(\mathsf{thmT}\,w)\,(\mathsf{Kcode}_{L^\star}\,(\mathsf{outKdef}_{L^\star}\,w))) \to\\ &\mathsf{Deriv}\,(\mathsf{eqF}\,(\mathsf{thmT}\,(\mathsf{cgFun}\,w))\,\mathsf{codeFalse}),\end{aligned}$$

whose proof is the second Σ-projection $\mathsf{cgFalse}\,w\,d := \mathsf{snd}\,(\mathsf{CGI\_core\_num\_raw\_self}\,w\,d)$, with the bridge $\mathsf{cgFun}\,w \equiv \mathsf{fst}\,(\mathsf{CGI\_core\_num\_raw\_self}\,w\,d)$ holding by refl thanks to Agda's let-substitution and record-projection unfolding.

3 The subtlety: internal evaluation semantics and the correctness proof

3.1 cgFun is a meta function, not a Fun_1

The reader might hope for $\mathsf{cgFun} : \mathsf{Fun}_1$, i.e. an element of the BRA syntactic algebra $\{s, o, u, C\}$ over $\mathsf{Fun}_2 = \{v, R\}$. This is provably *not possible* for the present construction. The Term $\mathsf{cgFun}\,w$

contains $\mathsf{closeW}\, w := \mathsf{substT}\, 0\, O\, (\mathsf{substT}\, 1\, O\, w)$, a meta-Agda function that pattern-matches on the constructor of w:

$$\mathsf{closeW}\, (\mathsf{var}\, 0) = O, \quad \mathsf{closeW}\, (\mathsf{var}\, 5) = \mathsf{var}\, 5.$$

A BRA $\mathsf{Fun}_1\, f$ over $\{s, o, u, C\}$ applied to w yields a Term $\mathsf{ap}_1\, f\, w$ whose structure is fixed by f and which can only *embed* w at fixed leaf positions; it cannot dispatch on w's head constructor. The two witnesses $w \in \{\mathsf{var}\, 0,\ \mathsf{var}\, 5\}$ already force any candidate to produce different shapes (O vs. $\mathsf{var}\, 5$), which no closed combinator tree achieves. The closeW step is therefore intrinsically meta-level.

The object-level substitution $\mathsf{sbt} : \mathsf{Fun}_2$ shipped in `BRA4.SbT` does not help: sbt acts on the Cantor *encoding* of Terms (firing on $\mathsf{ap}_2\, \mathsf{Pair}\, (\mathsf{natCode}\, \mathsf{tag\_var})\, (\mathsf{natCode}\, k)$), not on raw $\mathsf{var}\, k$).

3.2 Why closeW is needed in the first place

The module `BRA4.StepU2MuCorrect.Construct` that internally proves the mu-loop run

$$\mathsf{runs\_mu} : \mathsf{Deriv}\big(\mathsf{iter}\, \mathsf{step}\, (\mathsf{cfgEV}\, (\mathsf{mcodeMu}\, (\mathsf{mcode1}\, g_F))\, x_{\mathrm{outer}}\, K_0)\, (\mathsf{fuel}) = \mathsf{cfgRT}\, k_{\max}\, K_0\big)$$

requires three meta-level closedness conditions on its $k_{\max}$ parameter:

$$k_{\max\,\_\mathsf{sub}_0} : (a : \mathrm{Term}) \to k_{\max}[0 := a] = k_{\max}$$
$$k_{\max\,\_\mathsf{sub}_1} : (a : \mathrm{Term}) \to k_{\max}[1 := a] = k_{\max}$$
$$k_{\max\,\_\mathsf{sim}} : (a\, b : \mathrm{Term}) \to k_{\max}[\![0 := a,\ 1 := b]\!] = k_{\max}$$

(all are *meta* Eq, not Deriv). These are consumed by eqSubst-transports inside the ruleIndNat-with-substituted-motive proof of runs_mu.

Since $k_{\max} = \mathsf{gRec}\, O\, (s\, (\mathsf{closeW}\, w))$, structural induction on w (in `BRA4.CloseW`) discharges all three conditions for $\mathsf{closeW}\, w$. Without closeW, the conditions fail at, e.g., $w := \mathsf{var}\, 0$.

3.3 The internal evaluation correctness chain

The Deriv-half cgFalse relies on closing the chain

$$\mathsf{evalU}\, (\mathsf{parse}\, \mathsf{gLnameDef})\, n_T \stackrel{?}{=} s\, (\mathsf{outKdef}_{L^\star}\, k_{\max})$$

through seven internal segments (`ChainKdef`): `stepU_at_evC`, `runs_mu`, `stepU_at_rtC1`, `stepU_at_evU`, `stepU_at_rtApp2`, `runs2` (Lift1 $\mathsf{outKdef}_{L^\star}$), `stepU_at_rtEmpty`, glued by `iter_add_T`'s fuel-composition lemma. This run is then internalised as the thmT-fact

$$\mathsf{thmT}\, \mathsf{cPos} \stackrel{\mathsf{Deriv}}{=} \mathsf{coderunProg}\, g_L\, n_T = s\, x'$$

via `Thm 13` (singulary). The negative leg $\mathsf{thmT}\, (\mathsf{cmp}\, \mathsf{outerWrap}\, \mathsf{cSize}) = \mathsf{cNeg}\, \mathsf{coderunProg}\, g_L\, n_T = s\, x'$ is obtained from the hypothesis on w by two thmT_at_sb substitutions (`passK`) followed by encoded_mp against `dLenStarDef`'s size atom. The clash is then a single encoded_mp / encoded_exfalso pair.

3.4 No consistency hypothesis

Chaitin's original argument concludes "T is inconsistent" (in the metatheory). The BRA4 formalisation strengthens this: the conclusion is an *internal* $\mathsf{Deriv}\, (\mathsf{thmT}\, (\mathsf{cgFun}\, w) = \mathsf{codeFalse})$, i.e. a BRA derivation that the constructed code $\mathsf{cgFun}\, w$ is a T-proof of $0 = 1$. No Con(T) premise is consumed, no isNat integrality predicate is consumed, and the whole self-referential firing condition is stated in num-raw form (the subject slot is $\mathsf{num}\, x'$ for an arbitrary Term x', no integrality required to align with the read-back).

4 Files

- `BRA4/ChaitinG1CoreNumRaw.agda` — CGI_core_num_raw (x-parametric form, the workhorse).
- `BRA4/ChaitinG1CoreNumRawSelf.agda` — CGI_core_num_raw_self (the x-free, self-referential form, one-line specialisation of the workhorse).
- `BRA4/CgFun.agda` — cgFun : Term → Term and cgFalse, the Σ-decomposition.
- `BRA4/CloseW.agda` — the three structural-induction lemmas $\mathsf{cl\_w\_sub}_0$, $\mathsf{cl\_w\_sub}_1$, cl_w_sim enabling the Construct closure conditions at $k_{\max}(w)$.

All files typecheck under `--safe --without-K --exact-split` with no holes and no postulates.

References

[1] G. J. Chaitin, "Computational complexity and Gödel's incompleteness theorem," *ACM SIGACT News*, no. 9, pp. 11–12, 1971.

[2] G. J. Chaitin, "Information-theoretic limitations of formal systems," *Journal of the ACM*, vol. 21, no. 3, pp. 403–424, 1974.